Jose Fabian

CS 499: Computer Science Capstone

Milestone Three: Enhancement Two — Algorithms and Data Structures Narrative

Artifact Description

The artifact for this enhancement is my Autonomous Navigation AI Agent, created for CS-370 (Current and Emerging Trends in Computer Science) Project Two. The artifact is a Deep Q-Learning agent that learns to navigate an 8 by 8 treasure maze, built with TensorFlow and Keras. It consists of two files enhanced in this milestone: TreasureMaze_QLearning_Agent.py, which builds the neural network and runs the training loop, and GameExperience.py, which implements the experience replay buffer the agent learns from.

Justification for Inclusion

I chose this artifact for the algorithms and data structures category because reinforcement learning is at the intersection of algorithmic design and data structure choice. The replay buffer is a data structure problem, and how episodes are stored, evicted, and sampled affects correctness and performance; the choice between a neural network and a simpler tabular approach is a direct algorithmic design tradeoff with real complexity implications. This artifact allowed me to implement both types of thinking in one enhancement instead of treating them separately. The artifact was improved in a number of concrete ways. In GameExperience.py, I fixed a fundamental problem: the training loop creates GameExperience with a target_model argument, but the original constructor never accepted one, so target_model was silently assigned to max_memory instead. This broke the memory eviction check and meant the target network was

never actually used, even though its entire purpose is to keep Q-learning's bootstrapped targets stable while the live model is still changing. I updated the constructor to accept and store the target model properly, and updated get_data to compute its Q-value targets from that target network instead of the live model that is still evolving. I also replaced the list-based replay memory, which evicted old episodes with the O(n) operation del self.memory[0], with a collections.deque of fixed maximum length that evicts in constant time. I vectorized the batch prediction logic in get_data so that it runs two batched prediction calls, one for the live model and one for the target model, instead of predicting one sample at a time in a loop, and I fixed a call-site bug where the training loop passed a batch_size keyword argument to a method whose parameter has always been named data_size, which would have led to a TypeError.

In TreasureMaze_QLearning_Agent.py, I removed Jupyter-notebook-only syntax so that the file runs as a standalone script, wrapped all executable logic in functions behind an if __name__ == "__main__": guard, and eliminated the module-level global epsilon variable and the bare model reference in train_step, both of which are now passed in explicitly instead of relied on as global state. I fixed a win-rate staleness bug where the printed win rate, and the epsilon-decay decision based on it, were calculated before the current epoch's result was added to the history, so they were always one epoch behind. I changed all magic numbers to named constants. The most useful thing that I did was add a simple tabular Q-learning agent to the maze and compare it to the DQN directly. This was the main algorithmic design tradeoff I saw in my review of Milestone One code: this maze is only 64 cells and 4 actions, which is small enough that having a full neural network with experience replay may not be required. Instead of simply claiming that as an opinion, I built the comparison and ran it. The tabular agent reached a 100 percent win rate in 222 epochs and roughly a quarter of a second of training time. I did not run the DQN to full

convergence for this comparison, as even a few epochs took over thirty seconds due to the overhead of a neural network forward and backward pass on every single step, but the per-epoch cost difference makes the tradeoff clear. This gave me a real, measured answer to a design question instead of a guess.

Course Outcomes

In my Module One plan, I identified this category as demonstrating algorithmic principles and design tradeoffs, specifically complexity analysis of the replay buffer and a comparison against a simpler algorithmic approach. This milestone met that goal directly. Fixing the O(n) list eviction to an O(1) deque is a concrete complexity improvement I can explain and justify, and building and actually running the tabular Q-learning comparison let me demonstrate, with real numbers rather than just a claim, that I understand when a more complex algorithm is and is not justified for a given problem.

This enhancement also ended up demonstrating more of the well-founded and innovative techniques and tools outcome than I originally scoped for this category, since fixing the target-network bug and eliminating global state both reflect standard, professional software engineering practice applied to a machine learning codebase. I do not have major updates to my outcome-coverage plan overall, but I would now describe this artifact as touching both the algorithms and data structures outcome and, more lightly, the software engineering outcome, similar to how my Milestone Two enhancement ended up touching the security mindset outcome more than originally planned.

Reflection

The best part of this process was to see that the target-network bug was real and not just a theoretical code-review question. When I read through this code for Milestone One, I knew the constructor mismatch was likely a bug based on reading the code really carefully, but I did not have a way to prove it without actually running it. Once I got TensorFlow installed and was able to actually run the training loop, that is how I confirmed it, and the original code would have run without a Python error but would have never used the target network at all, precisely the kind of bug that is dangerous because it does not crash at all. It just quietly makes the model worse at what it is supposed to do. In the abstract, there is a difference between a bug that crashes and a bug that quietly degrades a system, and that is something I understood conceptually before this milestone but now understand firsthand.

The most difficult part was to explain the tabular-versus-DQN design tradeoff in a way that was honest and not just a way to make my own conclusion look good. It would have been very easy to only run the tabular agent and state that it is faster than the DQN without providing a fair comparison. Instead, I built both agents to share the same environment interface and ran them under comparable conditions. I did not run the DQN to full convergence, since its cost per epoch is much higher than the tabular agent's, and I was careful not to overstate the comparison because of that. This is a better skill than proving I can write two different algorithms, and I was glad to be able to present a design tradeoff in an honest manner, including its limitations.